



Semantica Vettoriale ed Embeddings

CORSO DI NATURAL LANGUAGE PROCESSING (ELABORAZIONE DEL LINGUAGGIO NATURALE)

a.a. 2026/2027

Prof. Roberto Pirrone



Semantica vettoriale

Modelli computazionali del significato delle parole

- Possiamo costruire una teoria su come **rappresentare il significato delle parole** che soddisfi almeno alcuni dei nostri desiderata?
- Introduciamo la ***semantica vettoriale***
- Il modello standard nell'elaborazione del linguaggio
- Raggiunge molti dei nostri obiettivi

Vector semantics: idea 1

- Wittgenstein, PI #43:
  "Il significato di una parola è il suo uso nel linguaggio"

Vector semantics: idea 1

- Definiamo “uso”:

  le parole sono definite dal loro “ambiente” (le parole attorno a loro)

- Zellig Harris (1954):

  «Se **A** e **B** hanno ambienti quasi identici, diciamo che sono **sinonimi**.»

Vector semantics: idea 1

- Supponiamo di leggere queste frasi:
  - *L'Ong choi è delizioso saltato con l'aglio.*
  - *L'Ong choi è superbo sul riso.*
  - *Le foglie di Ong choi con salse salate …*

- E di evere già visto queste:
  - *…spinaci saltati con l'aglio sul riso*
  - *I gambi e le foglie di bietola sono deliziosi*
  - *Il cavolo riccio e altre verdure a foglia verde salate …*

Vector semantics: idea 1

- Conclusione:
  - *Ong choi* è una **verdura a foglia verde**, simile agli spinaci, alla bietola o al cavolo riccio.
  - Possiamo giungere a questa conclusione grazie a parole come **“foglie”**, **“delizioso”** e **“saltato”**.

Ong choi



Idea 2: Il significato come un punto in uno spazio (Osgood et al. 1957)

- Ad esempio le tre dimensioni affettive di una parola
  - **valence**: piacevolezza
  - **arousal**: intensità dell'emozione
  - **dominance**: grado di controllo esercitato

- Da qui la rappresentazione come un vettore (in questo caso tridimensionale)

Definire il significato come un punto nello spazio in base alla distribuzione linguistica.

- Ogni parola → un vettore (non semplicememte "good" o "w_{45}")
- Parole simili sono ***vicine nello spazio semantico***
- Costruiamo questo spazio ***in modo automatico***, osservando ***quali parole compaiono vicine nel testo***.



Definiamo il significato di una parola come un vettore

- Chiamato ***embedding*** perché è *inserito (embedded)* all'interno di uno spazio
- È il ***metodo standard per rappresentare il significato*** in NLP.

  👉 Ogni moderno algoritmo di NLP utilizza gli embedding come rappresentazione del significato delle parole.
- Utile per analizzare le ***similarità*** tra parole.

Perché vettori?

- Consideriamo la sentiment analysis:
  - Usando direttamente le **parole**, una *feature* è semplicemente l'identità della parola o una regola
  - Esempio - Feature 5: la parola precedente è "terrible"
  - È necessario che *esattamente la stessa parola* compaia nel training set e nel test set

Perché vettori?

- Consideriamo la sentiment analysis:
  - Con gli embeddings:
    - La feature è un word vector, ad esempio: il vettore [35,22,17…]
    - Potremmo trovare una parola *simile* nel test set: [34,21,14, …]
    - Quindi possiamo ***generalizzare a parole simili*** non osservate prima!!!

Embedding TF-IDF

Affronteremo due tipi di embeddings

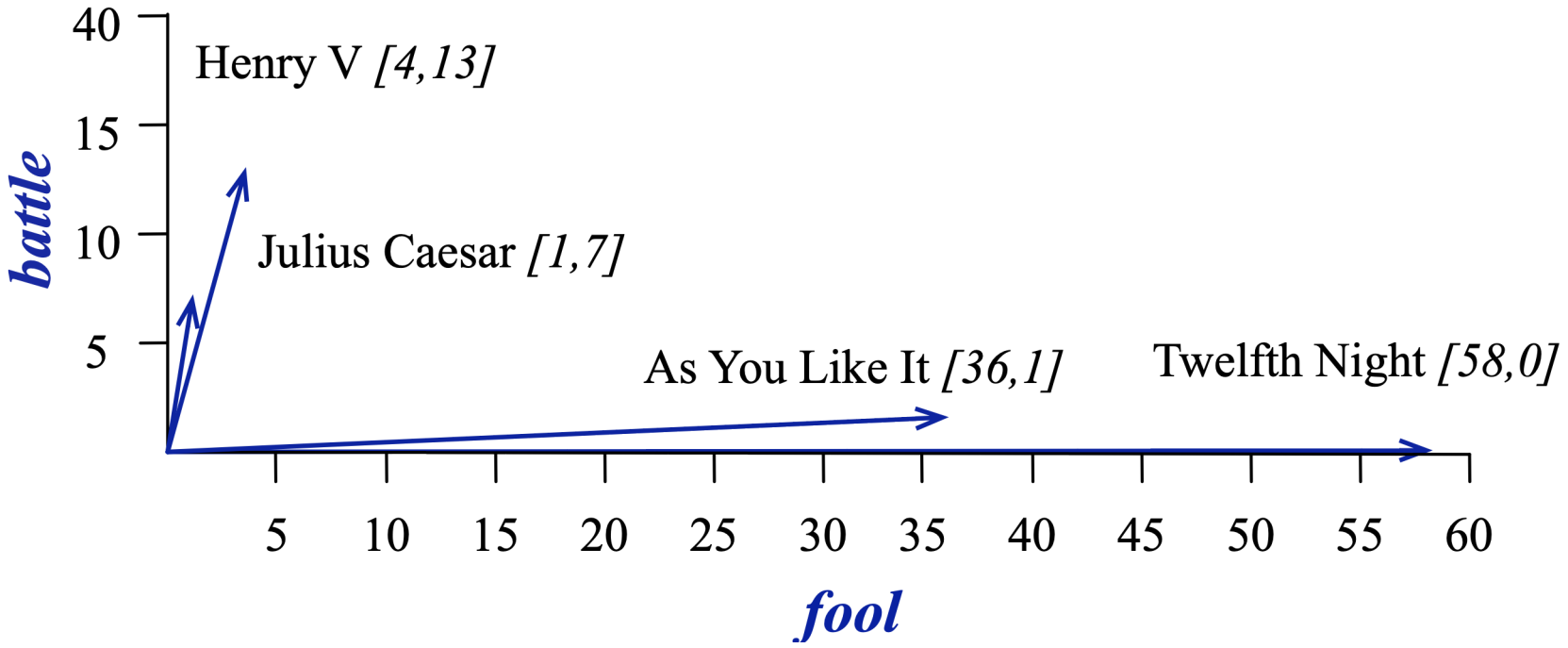
- tf-idf
  - Tipico dell'Information Retrieval
  - Baseline comune per molti approcci
  - Vettori ***sparsi***
  - Le parole sono rappresentate da **(una semplice funzione dei)** conteggi delle **parole vicine** nel testo.
- Word2vec
  - Vettori ***densi***
  - La rappresentazione viene creata addestrando un classificatore a ***prevedere se una parola è probabile che compaia nelle vicinanze***.
  - Esiste un'estensione chiamata **contextual embeddings**

Term-document matrix

Ogni documento è rappresentato come un vettore di parole

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Visualizzazione dei vettori dei documenti



I vettori sono la base dell'Information Retrieval

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

I vettori sono simili per le due commedie

Ma le commedie sono differenti dagli altri due documenti

Hanno più *fools* e *wit* e meno *battles*.

Ma anche le parole possono essere vettori!

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

battle è “quel tipo di parola che occorre in Giulio Cesare ed Enrico V”

fool è “quel tipo di parola che occore nelle commedie, specialmente La Dodicesima Notte”

Più comunememte: word-word matrix (o "term-context matrix")

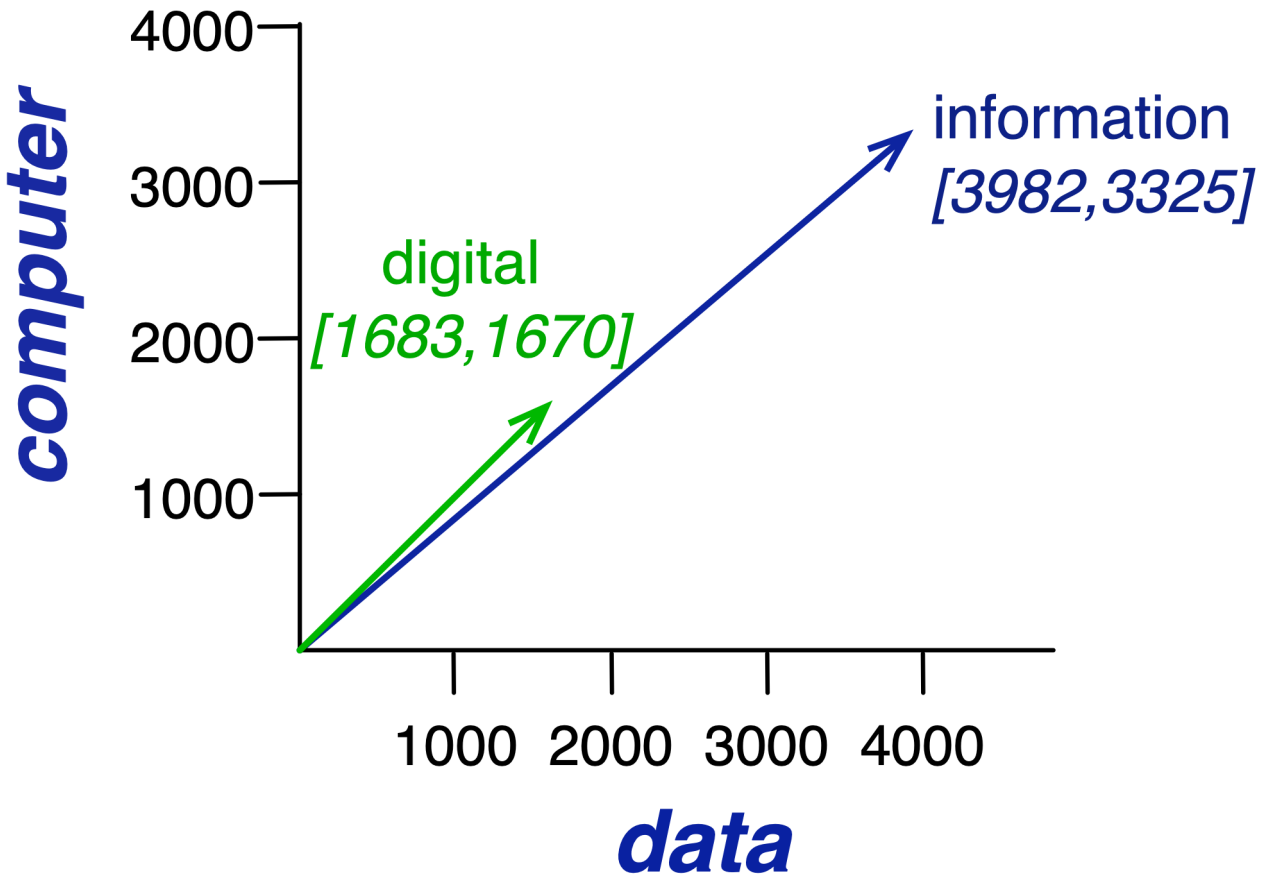
Un documento o una finestra di parole (per es. ±4) attorno alla parola data

- Due parole sono simili se i loro context vector lo sono

is traditionally followed by **cherry** pie, a traditional dessert
often mixed, such as **strawberry** rhubarb pie. Apple pie
computer peripherals and personal **digital** assistants. These devices usually
a computer. This includes **information** available on the internet

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Esempio di visualizzazione dei context vector



Calcoliamo la similarità tra parole: prodotto scalare e similarità coseno

- Il prodotto scalare tra due vettori è uno scalare:

$$\text{dot product}(\mathbf{v},\mathbf{w}) = \mathbf{v}\cdot\mathbf{w} = \sum_{i=1}^{N} v_i w_i = v_1 w_1 + v_2 w_2 + \ldots + v_N w_N$$

- Tende a crescere quando I due vettori hanno grandi valori lungo le stesse dimensioni

- Può essere una buona metrica di similarità tra vettori

Problemi del prodotto scalare

- Il prodotto scalare favorisce I vettori lunghi infatti è tanto più elevato quanto il vettore è lungo cioè ha valori alti lungo molte dimensioni

$$|\mathbf{v}| = \sqrt{\sum_{i=1}^{N} v_i^2}$$

- Le parole frequenti come le stopwords (of, the, you) hanno vettori lunghi perché co-occorrono molte volte con le altre parole.
- Il prodotto scalare favorisce troppo le parole frequenti

La similarità coseno come alternativa

$$\text{cosine}(\vec{v},\vec{w}) = \frac{\vec{v}\cdot\vec{w}}{|\vec{v}||\vec{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$

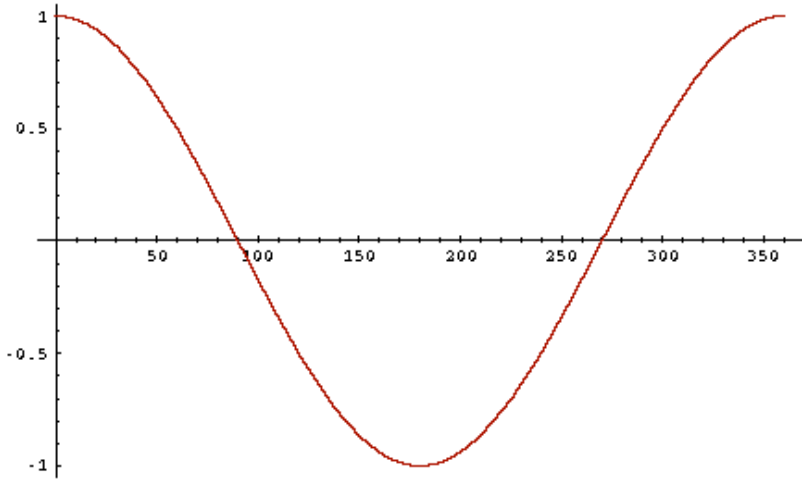
E’ basata sulla definizione di prodotto scalare

$$\mathbf{a}\cdot\mathbf{b} = |\mathbf{a}||\mathbf{b}|\cos\theta$$

$$\frac{\mathbf{a}\cdot\mathbf{b}}{|\mathbf{a}||\mathbf{b}|} = \cos\theta$$

Similarità coseno

- +1: per vettori che puntano **nella stessa direzione**
- -1: per vettori che puntano **in direzioni opposte**
- 0: per vettori **ortogonali**



Tuttavia, poiché i valori di frequenza grezzi sono non negativi, il coseno per questi vettori varia effettivamente da 0 a 1

Esempi

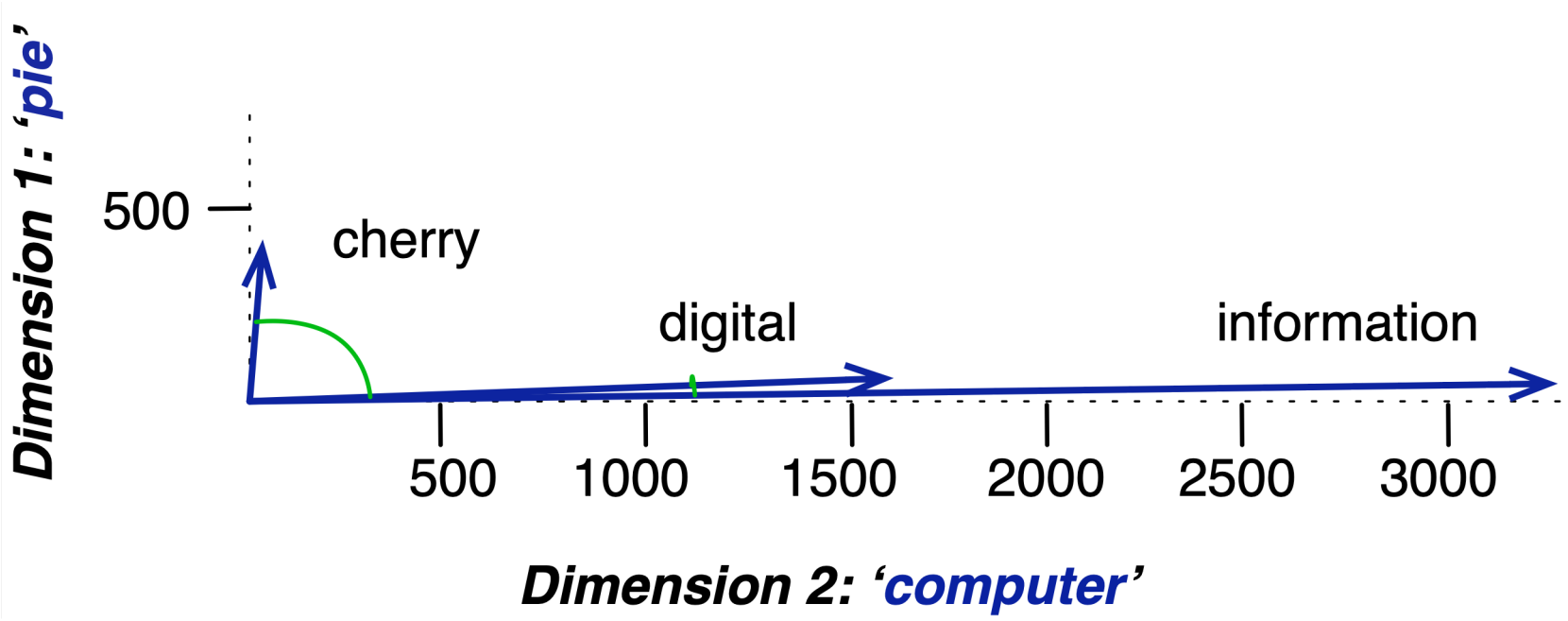
$$\cos(\vec{v},\vec{w}) = \frac{\vec{v}\bullet\vec{w}}{|\vec{v}||\vec{w}|} = \frac{\vec{v}}{|\vec{v}|}\bullet\frac{\vec{w}}{|\vec{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

$$\cos(\text{cherry},\text{information}) = \frac{442*5+8*3982+2*3325}{\sqrt{442^2+8^2+2^2}\sqrt{5^2+3982^2+3325^2}} = .017$$

$$\cos(\text{digital},\text{information}) = \frac{5*5+1683*3982+1670*3325}{\sqrt{5^2+1683^2+1670^2}\sqrt{5^2+3982^2+3325^2}} = .996$$

Visualizziamo i coseni o meglio gli “angoli”



La frequenza grezza non è una buona rappresentazione

- Le matrici di co-occorrenza che abbiamo visto finora rappresentano ogni cella tramite le ***frequenze delle parole***.
- La frequenza è certamente utile: se *sugar* compare spesso vicino a *apricot*, questa è un'informazione rilevante.
- Tuttavia, parole eccessivamente frequenti come *the*, *it* o *they* non sono molto informative riguardo al contesto.
- come possiamo bilanciare questi due vincoli in conflitto?

Due possibili soluzioni

- **tf-idf:** il valore di tf-idf per la parola *t* nel documento *d* è:

$$w_{t,d} = \mathrm{tf}_{t,d} \times \mathrm{idf}_t$$

Parole come "the" or "it" hanno idf molto bassa

- **PMI:** (Pointwise mutual information)

$$\mathbf{PMI}(w_1, w_2) = log \frac{p(w_1,w_2)}{p(w_1)p(w_2)}$$

Verifichiamo se parole come ad es. "good" appaiono molto spesso con "great" di quanto potremmo aspettarci per caso

Term frequency (tf)

- $tf_{t,d} = count(t,d)$

- Invece di usare direttamente il conteggio, applichiamo il logaritmo:

$$tf_{t,d} = \log_{10}(count(t,d)+1)$$ *Sempre >= 0*

Applichiamo una scala logaritmica al conteggio grezzo perché non è direttamente collegato al significato del documento.

Document frequency (df)

- df_t è il numero di documenti in cui appare *t*.
- *Non è il conteggio totale della parola su tutti I documenti*
- "*Romeo*" molto discriminativo per una sola opera di Shakespeare:

	Collection Frequency	Document Frequency
Romeo	113	1
action	113	31

Inverse document frequency (idf)

$$\text{idf}_t = \log_{10}\left(\frac{N}{\text{df}_t}\right)$$

Sempre >= 0

N è il lnumero totale di documenti nella collezione

Word	df	idf
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.967
forest	12	0.489
battle	21	0.246
wit	34	0.037
fool	36	0.012
good	37	0
sweet	37	0

Cos’è un documento?

- Qualunque unità di testo

- Normalmente ci si riferisce ad un paragrafo come a un documento

Calcolo finale del tf-idf

$$w_{t,d} = \mathrm{tf}_{t,d} \times \mathrm{idf}_t$$

- Raw counts:

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

- tf-idf:

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.074	0	0.22	0.28
good	0	0	0	0
fool	0.019	0.021	0.0036	0.0083
wit	0.049	0.044	0.018	0.022

Pointwise Mutual Information

- **Pointwise mutual information**:

  Gli eventi x e y occorrono più spesso che se fossero statisticamente indipendenti?

$$\mathrm{PMI}(X,Y) = \log_2 \frac{P(x,y)}{P(x)P(y)} \qquad MI(X,Y) = \sum_x \sum_y P(x,y) \log_2 \frac{P(x,y)}{P(x)P(y)} =$$

$$= \mathbb{E}_{x\sim X, y\sim Y} \left[PMI(x,y)\right]$$

Tra due parole: (Church & Hanks 1989)

le parole w1 e w2 co-occorrono di più che se fossero indipendenti?

$$\mathrm{PMI}(word_1, word_2) = \log_2 \frac{P(word_1, word_2)}{P(word_1)P(word_2)}$$

Positive Pointwise Mutual Information

- PMI varia tra $-\infty$ e $+\infty$
- I valori negativi sono problematici
  - PMI < 0: la co-occorrenza è ***meno*** di quanto ci aspetteremmo per caso
  - Inaffidabile senza corpora enormi
- I valori negativi di PMI vengono sostituiti da 0
- Positive PMI (**PPMI**) between word1 and word2:

$$\text{PPMI}(word_1, word_2) = \max\left(\log_2 \frac{P(word_1, word_2)}{P(word_1)P(word_2)}, 0\right)$$

Calcoliamo la PPMI su una term-context matrix

- Matrice F con W righe (parole) e C colonne (contesti)
- f_{ij} è il numero di volte in cui w_i occorre nel contesto c_j

$$p_{ij} = \frac{f_{ij}}{\sum_{i=1}^{W}\sum_{j=1}^{C} f_{ij}} \qquad p_{i*} = \frac{\sum_{j=1}^{C} f_{ij}}{\sum_{i=1}^{W}\sum_{j=1}^{C} f_{ij}} \qquad p_{*j} = \frac{\sum_{i=1}^{W} f_{ij}}{\sum_{i=1}^{W}\sum_{j=1}^{C} f_{ij}}$$

	computer	data	result	pie	sugar	count(w)
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
count(context)	4997	5673	473	512	61	11716

$$pmi_{ij} = \log_2 \frac{p_{ij}}{p_{i*}p_{*j}} \qquad ppmi_{ij} = \begin{cases} pmi_{ij} & \text{if } pmi_{ij} > 0 \\ 0 & \text{otherwise} \end{cases}$$

Università degli Studi di Palermo

dipartimento di ingegneria unipa

CHILAB

$$p_{ij} = \frac{f_{ij}}{\sum_{i=1}^{W}\sum_{j=1}^{C} f_{ij}}$$

	computer	data	result	pie	sugar	count(w)
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
count(context)	4997	5673	473	512	61	11716

- $p(w=\text{information}, c=\text{data}) = 3982/11716 = .3399$
- $p(w=\text{information}) = 7703/11716 = .6575$
- $p(c=\text{data}) = 5673/11716 = .4842$

$$p(w_i) = \frac{\sum_{j=1}^{C} f_{ij}}{N} \qquad p(c_j) = \frac{\sum_{i=1}^{W} f_{ij}}{N}$$

	p(w,context)					p(w)
	computer	**data**	**result**	**pie**	**sugar**	**p(w)**
cherry	0.0002	0.0007	0.0008	0.0377	0.0021	0.0415
strawberry	0.0000	0.0000	0.0001	0.0051	0.0016	0.0068
digital	0.1425	0.1436	0.0073	0.0004	0.0003	0.2942
information	0.2838	0.3399	0.0323	0.0004	0.0011	0.6575
p(context)	0.4265	0.4842	0.0404	0.0437	0.0052	

Università degli Studi di Palermo
dipartimento di ingegneria unipa
CHILAB

$$pmi_{ij} = \log_2 \frac{p_{ij}}{p_{i*}p_{*j}}$$

	p(w,context)					p(w)
	computer	**data**	**result**	**pie**	**sugar**	**p(w)**
cherry	0.0002	0.0007	0.0008	0.0377	0.0021	0.0415
strawberry	0.0000	0.0000	0.0001	0.0051	0.0016	0.0068
digital	0.1425	0.1436	0.0073	0.0004	0.0003	0.2942
information	0.2838	0.3399	0.0323	0.0004	0.0011	0.6575
p(context)	0.4265	0.4842	0.0404	0.0437	0.0052	

- pmi(information,data) = $\log_2$ (.3399 / (.6575*.4842)) = .0944

Resulting PPMI matrix (negatives replaced by 0)

	computer	**data**	**result**	**pie**	**sugar**
cherry	0	0	0	4.38	3.30
strawberry	0	0	0	4.10	5.51
digital	0.18	0.01	0	0	0
information	0.02	0.09	0.28	0	0

Weighting PMI

- PMI polarizzata verso gli eventi infrequenti
  - Parole molto rare hanno valori di PMI molto elevati

- Due soluzioni:
  - Dare alle parole rare probabilità un po' più alte
  - Usare lo smoothing add-1 che ha un effetto simile

Pesare la PMI

- Elevare le probabilità di contesto ad $\alpha = 0.75$:

$$\text{PPMI}_\alpha(w,c) = \max(\log_2 \frac{P(w,c)}{P(w)P_\alpha(c)}, 0)$$

$$P_\alpha(c) = \frac{count(c)^\alpha}{\sum_c count(c)^\alpha}$$

- Ci aiuta perché $P_\alpha(c) > P(c)$ per *c* raro
- Facciamo un esempio con P(a) = .99 and P(b)=.01
- $P_\alpha(a) = \frac{.99^{.75}}{.99^{.75}+.01^{.75}} = .97 \quad P_\alpha(b) = \frac{.01^{.75}}{.99^{.75}+.01^{.75}} = .03$

Vettori sparsi vs vettori densi

- I vettori tf-idf o PMI sono
  - **lunghi** (dimensione |V| tra 20.000 e 50.000)
  - **sparsi** (la maggior parte degli elementi sono nulli)
- In alternativa ***apprendiamo*** vettori che sono
  - **corti** (lunghezza tra 50 e 1000)
  - **densi** (la maggior parte degli elementi sono non nulli)

Vettori sparsi vs vettori densi

- Perché vettori densi e corti?
  - Sono più facili da usare nei modelli di machine learning: ci sono meno parametri da apprendere
  - Generalizzano meglio dei conteggi espliciti
  - Catturano meglio la sinonimia
    - *car* e *automobile* sono sinonimi, ma sono dimensioni diverse
    - Una parola vche ha *car* come vicina e una che ha come vicina *automobile* potrebbero essere simili, ma in generale non lo sono

Word2vec

Metodi comuni per ottenere vettori corti e densi

- Ispirati dal NLP
  - Word2vec (skipgram, CBOW), GloVe
- Singular Value Decomposition (SVD)
  - Un caso speciale di SVD è la LSA (Latent Semantic Analysis)
- In alternativa a questi “embedding statici":
  - Contextual Embeddings (ELMo, BERT)
  - Calcoliamo embedding distinti per la stessa parola in ogni suo contesto
  - Calcoliamo embedding idstinti per ogni token della parola

Semplici embedding statici che si possono scaricare

- Word2vec (Mikolov et al)
- https://code.google.com/archive/p/word2vec/

- GloVe (Pennington, Socher, Manning)
- http://nlp.stanford.edu/projects/glove/

Word2vec

- Metodo di embeddingmolto popolare
  - Molto veloce da addestrare
  - Codice disponibile sul web

- **Idea**: *predire* invece di contare

- Word2vec offre varie opzioni

  Noi useremo: ***skip-gram with negative sampling (SGNS)***

Word2vec

- Invece di contare quante volte ogni parola *w* compare vicino ad «albicocca» addestriamo un classificatore su un task di predizione binaria:
  - È probabile che *w* compaia vicino ad "albicocca"?
- In realtà non ci interessa questo task, ma useremo i pesi appresi dal classificatore come word embedding
- Grande idea: ***self-supervision***
  - Una parola *c* che compare vicino ad "albicocca" nel corpus funge da "risposta corretta" (gold standard) per l'apprendimento supervisionato
  - Non c'è bisogno di etichette umane

Bengio et al. (2003); Collobert et al. (2011)

Approccio: predire se una parola candidata *c* è un "vicino"

1. Trattiamo la parola target *t* e una parola di contesto vicina *c* come *esempi positivi*
2. Campioniamo casualmente altre parole nel lessico per ottenere *esempi negativi*
3. Usiamo la regressione logistica per addestrare un classificatore a distinguere i due casi
4. Usiamo i pesi appresi come embedding

Dati di training per Skip-Gram

- Assumiamo una finestra di +/- 2 parole, data la frase di training:

…lemon, a [tablespoon of apricot jam, a] pinch…

c1 c2 [target] c3 c4

Classificatore Skip-Gram

- assumendo una finestra di +/- 2 parole

…lemon, a [tablespoon of apricot jam, a] pinch…
c1 c2 [target] c3 c4

- **Obiettivo**: addestrare un classificatore che, data una coppia candidata (parola, contesto)

(apricot, jam)
(apricot, aardvark)
…

- Assegni ad ogni coppi auna probabilità:

$$P(+|w, c)$$
$$P(-|w, c) = 1 - P(+|w, c)$$

La similarità è calcolata dal prodotto scalare

- Due vettori sono simili se hanno un prodotto scalare elevato
  - Il coseno è solo un prodotto scalare normalizzato
- Di conseguenza:
  - $Similarità(w,c) \propto w \cdot c$
- Dovremo normalizzare per ottenere una probabilità
  - anche il coseno non è una probabilità

Modificare i prodotti scalari in probabilità

- *Similarità*(*w*,*c*) $\propto w \cdot c$
- Per trasformarlo in una probabilità useremo la sigmoide dalla regressione logistica:

$$P(+|w,c) = \sigma(c \cdot w) = \frac{1}{1+\exp(-c \cdot w)}$$

$$P(-|w,c) = 1 - P(+|w,c)$$

$$= \sigma(-c \cdot w) = \frac{1}{1+\exp(c \cdot w)}$$

Come il classificatore Skip-Gram calcola P(+|w, c)

$$P(+|w,c) = \sigma(c \cdot w) = \frac{1}{1+\exp(-c \cdot w)}$$

- Questo vale per una parola di contesto, ma abbiamo molte parole di contesto.
- Assumeremo l'indipendenza statistica e semplicemente le moltiplicheremo:

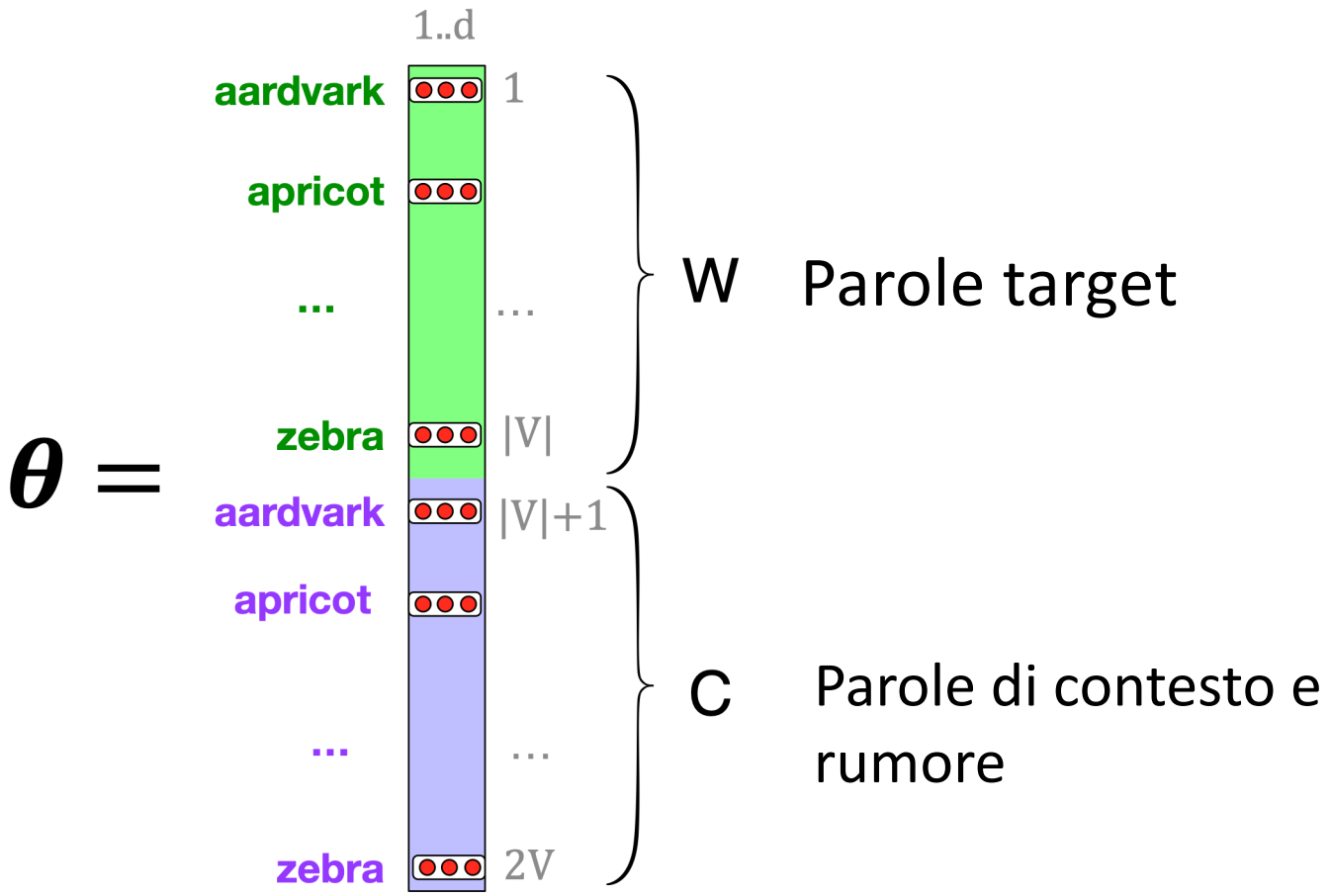
$$P(+|w,c_{1:L}) = \prod_{i=1}^{L} \sigma(c_i \cdot w)$$

$$\log P(+|w,c_{1:L}) = \sum_{i=1}^{L} \log \sigma(c_i \cdot w)$$

Riassumendo

- Un classificatore probabilistico che, data una parola target di test *w* e la sua finestra di contesto di *L* parole $c_{1:L}$
- Stima la probabilità che *w* compaia in questa finestra basandosi sulla similarità degli embedding di *w* con gli embedding di $c_{1:L}$.
- Per calcolare questo, abbiamo solo bisogno degli embedding per tutte le parole.

Avremo bisogno di un set di embedding per *w* ed uno per *c*



Dati di addestramento di Skip-Gram

…lemon, a [tablespoon of apricot jam, a] pinch…

c1 c2 [target] c3 c4



Dati di addestramento di Skip-Gram

…lemon, a [tablespoon of apricot jam, a] pinch…

c1 c2 [target] c3 c4



Per ogni esempio positivo, prenderemo *k* esempi negativi, campionando per frequenza

Dati di addestramento di Skip-Gram

…lemon, a [tablespoon of apricot jam, a] pinch…

c1 c2 [target] c3 c4



negative examples -

t	c	t	c
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

Word2vec: come apprendere i vettori

- Dato l'insieme di istanze di training positive e negative, e un set iniziale di vettori di embedding
- L'obiettivo dell'apprendimento è aggiustare quei vettori di parole in modo da:
  - ***Massimizzare*** la similarità delle coppie (w, c_{pos}) estratte dai dati positivi
  - ***Minimizzare*** la similarità delle coppie (w, c_{neg}) estratte dai dati negativi.

Funzione di Loss per una parola *w con* c_{pos} , c_{neg1} …c_{negk}

- Massimizzare la similarità del target con le parole di contesto reali, e minimizzare la similarità del target con le k parole non-vicine campionate negativamente.

$$
\begin{aligned}
L_{CE} &= -\log\left[P(+|w,c_{pos})\prod_{i=1}^{k}P(-|w,c_{neg_i})\right] \\
&= -\left[\log P(+|w,c_{pos})+\sum_{i=1}^{k}\log P(-|w,c_{neg_i})\right] \\
&= -\left[\log P(+|w,c_{pos})+\sum_{i=1}^{k}\log\left(1-P(+|w,c_{neg_i})\right)\right] \\
&= -\left[\log\sigma(c_{pos}\cdot w)+\sum_{i=1}^{k}\log\sigma(-c_{neg_i}\cdot w)\right]
\end{aligned}
$$

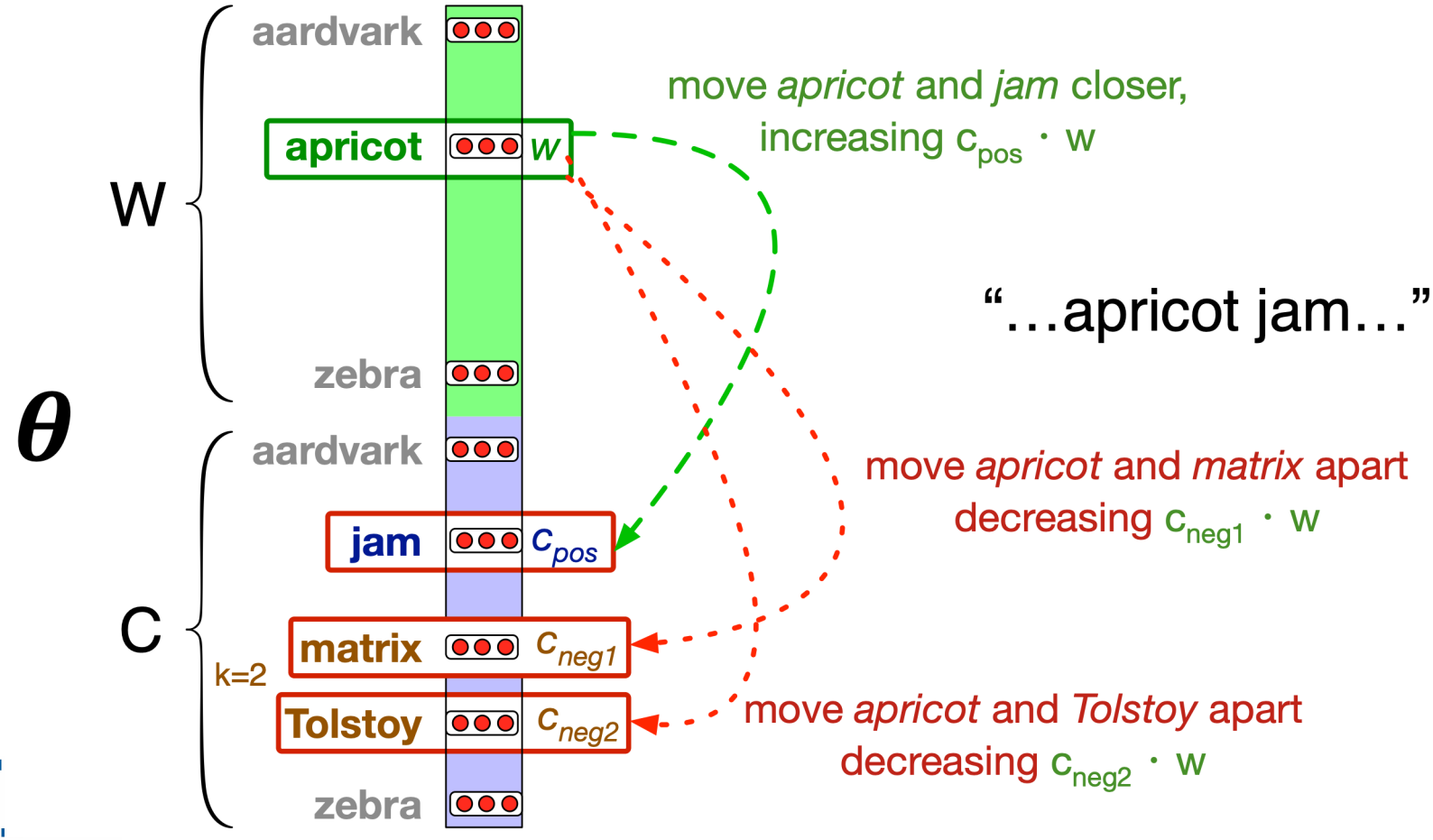
Apprendimento del classificatore

- Come apprendiamo?

  Stochastic gradient descent!

- Troveremo Il set di pesi per rendere le coppie positive più probabili e quelle negative meno probabili sull'intero training set.

Singolo passo del Gradient Descent



Gradient descent

- Ad ogni passo:
  - **Direzione**: Ci muoviamo nella direzione opposta al gradiente della funzione di perdita
  - **Magnitudine**: Muoviamo il valore di questo gradiente $\frac{d}{dw} L(f(x;w),y)$ pesato da un ***learning rate*** η

  - Un learning rate più alto significa muovere *w* più velocemente

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x;w),y)$$

Derivate della funzione di Loss

$$L_{CE} = -\left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^{k} \log \sigma(-c_{neg_i} \cdot w)\right]$$

$$\frac{\partial L_{CE}}{\partial c_{pos}} = [\sigma(c_{pos} \cdot w) - 1]w$$

$$\frac{\partial L_{CE}}{\partial c_{neg}} = [\sigma(c_{neg} \cdot w)]w$$

$$\frac{\partial L_{CE}}{\partial w} = [\sigma(c_{pos} \cdot w) - 1]c_{pos} + \sum_{i=1}^{k} [\sigma(c_{\mathrm{neg}_i} \cdot w)]c_{neg_i}$$

Aggiornamento in SGD

Iniziamo con le matrici *C* e *W* inizializzate casualmente, quindi eseguiamo aggiornamenti incrementali.

$$
\begin{aligned}
c_{pos}^{t+1} &= c_{pos}^{t} - \eta[\sigma(c_{pos}^{t} \cdot w^{t}) - 1]w^{t} \\
c_{neg}^{t+1} &= c_{neg}^{t} - \eta[\sigma(c_{neg}^{t} \cdot w^{t})]w^{t} \\
w^{t+1} &= w^{t} - \eta \left[[\sigma(c_{pos} \cdot w^{t}) - 1]c_{pos} + \sum_{i=1}^{k} [\sigma(c_{\mathrm{neg}_i} \cdot w^{t})]c_{neg_i} \right]
\end{aligned}
$$

Due insiemi di embedding

- Il modello SGNS apprende ***due insiemi di embedding***:
  - la matrice degli embedding target (*W*)
  - la matrice degli embedding context (*C*)

È pratica comune *sommarli*, rappresentando la parola *i* con il vettore $w_i + c_i$.

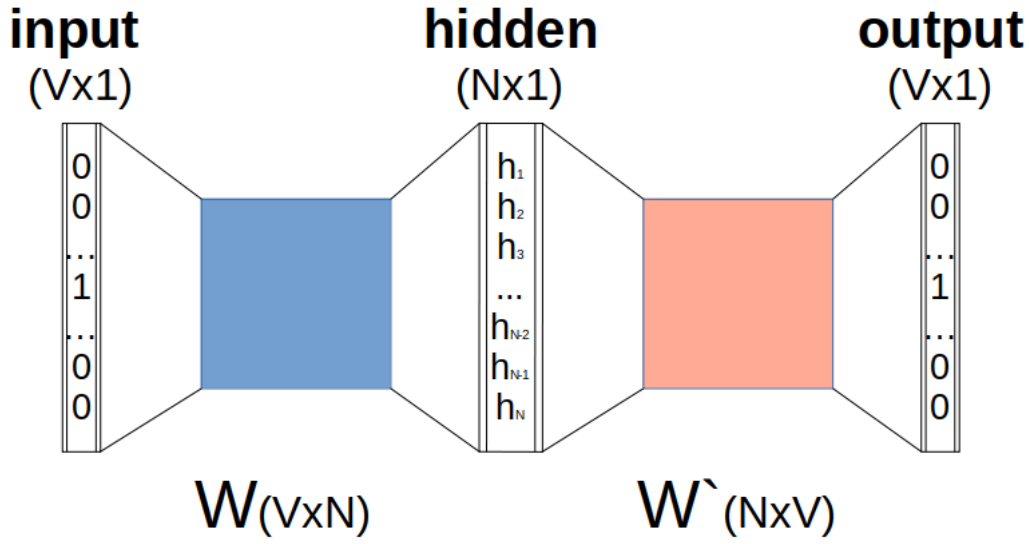
Riassumendo: apprendimento degli embedding Word2vec Skip-gram

- Iniziamo con *V* vettori casuali *d*-dimensionali come embedding iniziali
- Addestriamo un classificatore basato sulla similarità degli embedding

1. Prendiamo un corpus e consideriamo le coppie di parole che co-occorrono come esempi positivi
2. Prendiamo coppie di parole che non co-occorrono come esempi negativi
3. Addestriamo un classificatore a distinguerle, aggiustando lentamente tutti gli embedding per migliorare le prestazioni del classificatore
4. Scartiamo il classificatore e teniamo gli embedding.

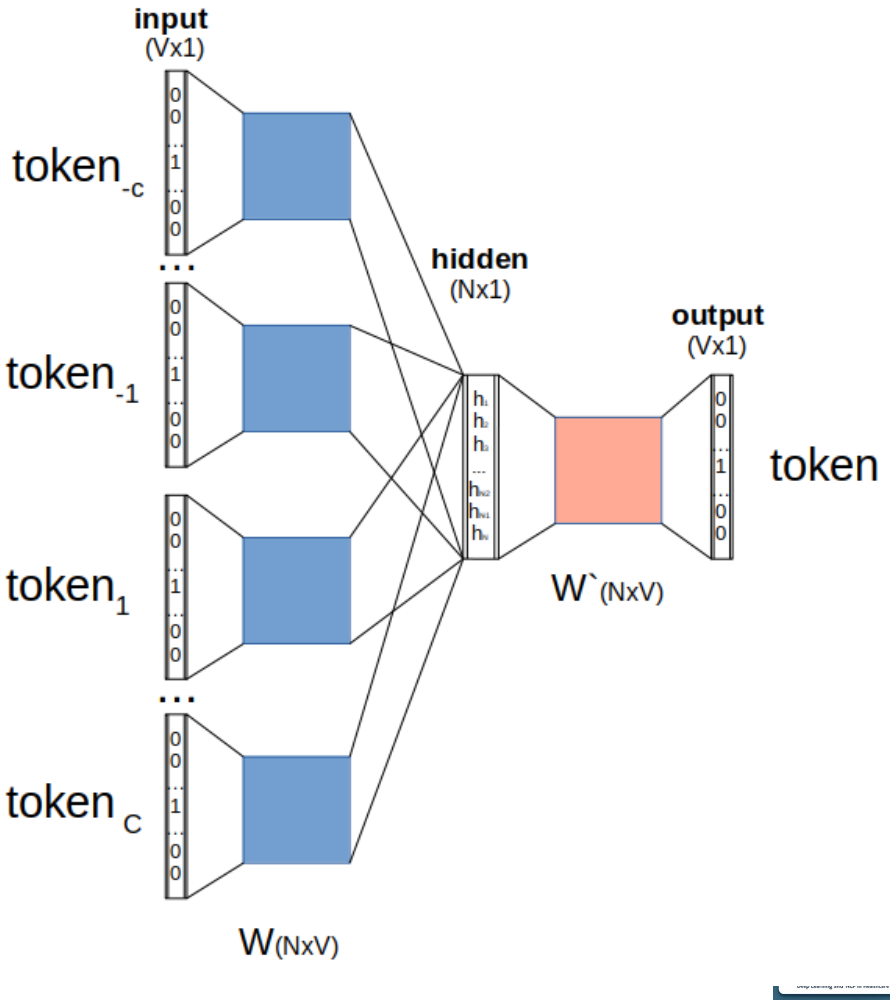
Skip-gram e CBOW come rete neurale

- Archietettura fully-connected generica
- One-hot encoding per ogni parola
- Vogliamo apprendere la matrice W
- Lo strato nascosto ha dimensione Nx1 per fornire la giusta dimensione a W



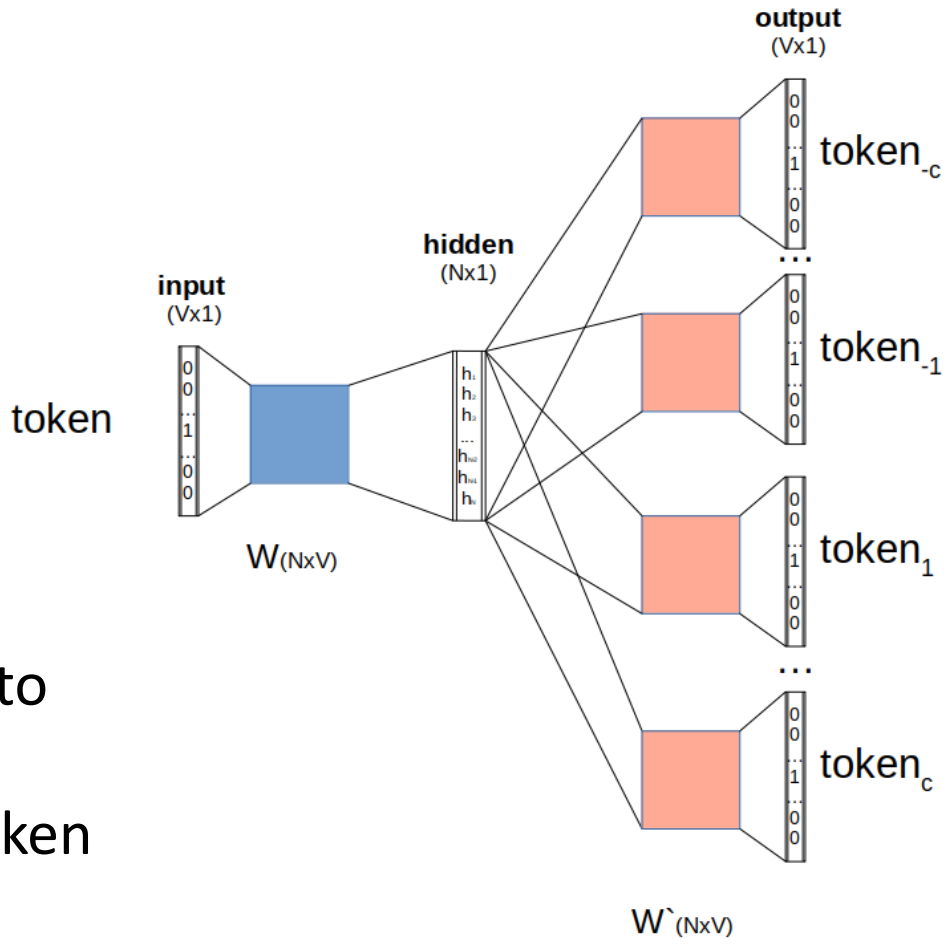
Skip-gram e CBOW come rete neurale

- CBOW – Context Bag Of Words
- Pediciamo la parola dal contesto
- La stessa matrice W per ogni token
- La riga W_i è l'embedding dell'*i*-esimo token
- Lo strato nascosto esegue una media degli ingressi, e quindi *perdiamo il positional encoding*



Skip-gram e CBOW come rete neurale

- Skip-gram
- Prediciamo il contesto dalla parola
- Un'unica matrice W
- Gli errori sulle uscite dei singoli token vengono sommati prima della backpropagation verso lo strato nascosto
- Lo strato nascosto è l'embedding del token

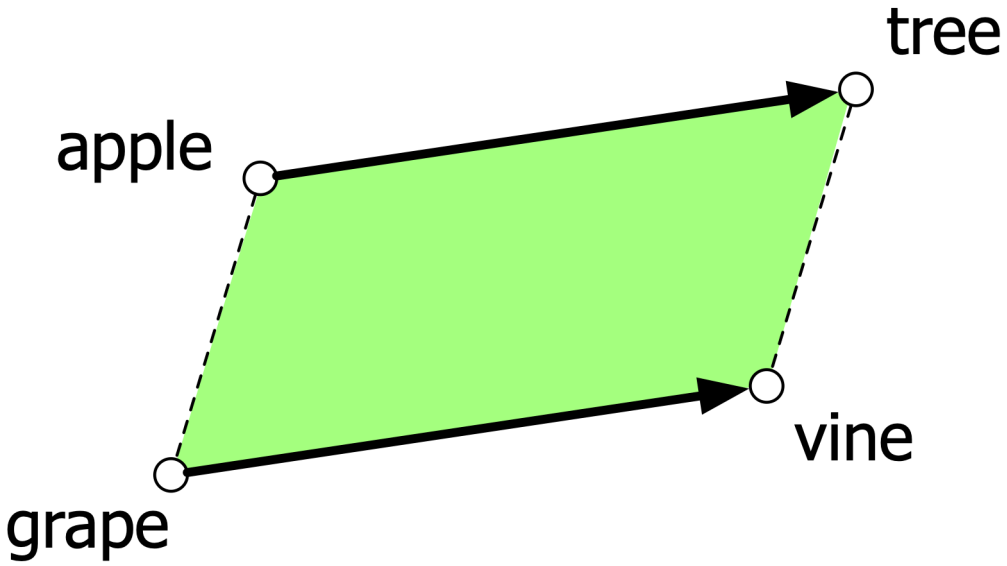


I tipi dei vicini dipendono dalla dimensione della finestra

- **Finestre piccole (C= +/- 2)**: le parole più vicine sono parole sintatticamente simili nella stessa tassonomia
  - I vicini più prossimi di **Hogwarts** sono altre scuole fittizie: *Sunnydale, Evernight, Blandings*
- **Finestre grandi (C= +/- 5)**: le parole più vicine sono parole correlate nello stesso campo semantico
  - I vicini più prossimi di **Hogwarts** sono parole del mondo di Harry Potter: *Silente, mezzosangue, Malfoy*

Relazioni di analogia

- Il modello classico del parallelogramma per il ragionamento per analogia (Rumelhart and Abrahamson 1973)
- Per risolvere: *"apple sta a tree come grape sta a _____"*
- *Sommiamo tree – apple a grape per ottenere vine*



Analogical relations via parallelogram

- Il metodo del parallelogramma può risolvere analogie sia con embedding sparsi sia con quelli densi (Turney and Littman 2005, Mikolov et al. 2013b)
- king – man + woman è vicino a queen
- Paris – France + Italy è vicino a Rome

- Per un problema a: $\mathbf{a} : \mathbf{b} :: \mathbf{a}^* : \mathbf{b}^*$ il metodo del parallelogramma calcola:

$$\hat{\mathbf{b}}^* = \underset{\mathbf{x}}{\operatorname{argmin}}\ \operatorname{distance}(\mathbf{x}, \mathbf{b} - \mathbf{a} + \mathbf{a}^*)$$

Avvertenze sul metodo del parallelogramma

- Sembra funzionare solo per parole frequenti, piccole distanze e certe relazioni (legare paesi a capitali, o parti del discorso), ma non per altre. (Linzen 2016, Gladkova et al. 2016, Ethayarajh et al. 2019a)

- Compendere l'analogia è ancora un campo di ricerca aperto (Peterson et al. 2020)

Embedding come una finestra sulla semantica storica

Addestriamo gli embedding su decenni diversi di testi storici per osservare il cambiamento dei significati

~30 milioni di libri dal 1850 al 1990, dati di Google Books



William L. Hamilton, Jure Leskovec, and Dan Jurafsky. 2016. Diachronic Word Embeddings Reveal Statistical Laws of Semantic Change. Proceedings of ACL.

Università degli Studi di Palermo

dipartimento di ingegneria unipa

CHILAB

Gli embedding riflettono I bias culturali!

Bolukbasi, Tolga, Kai-Wei Chang, James Y. Zou, Venkatesh Saligrama, and Adam T. Kalai. "Man is to computer programmer as woman is to homemaker? debiasing word embeddings." In *NeurIPS*, pp. 4349-4357. 2016.

- Se chiediamo “Paris : France :: Tokyo : x”
  - x = Japan
- Se chiediamo “father : doctor :: mother : x”
  - x = nurse
- Se chiediamo “man : computer programmer :: woman : x”
  - x = homemaker

Algoritmi che usano embedding, ad esempio, nelle ricerche di personale per programmatori, potrebbero portare a bias nelle assunzioni

Fasttext e GloVe

fasttext (Bojanowski et al., 2017)

- word2vec non gestisce bene le partole sconosciute cioè quelle che appaiono nel test set, ma non nel training set
- Si parla di «word sparsity»
- Linguaggi con una morfologia ricca in cui alcune delle tante forme dei nomi e dei verbi occorrono raramente

fasttext

- fasttext gestisce questi problemi usando modelli a subword

- Ogni parola è rapresentata da sé stessa più una *bag* di n-grammi costituenti, cui vengono aggiunti i simboli speciali ‘<‘ e ‘>’ per indicare le terminazioni
- *n* = 3, *where* →

  `<where>, <wh, whe, her, ere, re>`

fasttext

- fasttext gestisce questi problemi usando modelli a subword
  - Ogni parola è rapresentata da sé stessa più una *bag* di n-grammi costituenti, cui vengono aggiunti i simboli speciali ‘<‘ e ‘>’ per indicare le terminazioni

1. Viene appreso un embedding skip-gram per ogni n-gramma costituente e per la parola
2. La parola è rappresentata dalla somma di tutti gli embedding dei suoi n-grammi costituenti

GloVe (Pennington et al.,2014)

- GloVe significa *Global Vectors*

- Il modello cerca di catturare le statistiche globali del corpus

- Un modello log-bilineare con una funzione obiettivo ai minimi quadrati pesati, addestrato sulle voci non nulle di una matrice globale di co-occorrenza parola-parola

GloVe (Pennington et al.,2014)

Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k\|ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k\|steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k\|ice)/P(k\|steam)$	8.9	8.5×10^{-2}	1.36	0.96

Le parole più discriminative hanno probabilità alte → rapporto >> 1

Le parole non discriminative hanno circa la stessa probabilità → rapporto ~ 1

Università degli Studi di Palermo

dipartimento di ingegneria unipa

CHILAB

GloVe (Pennington et al.,2014)

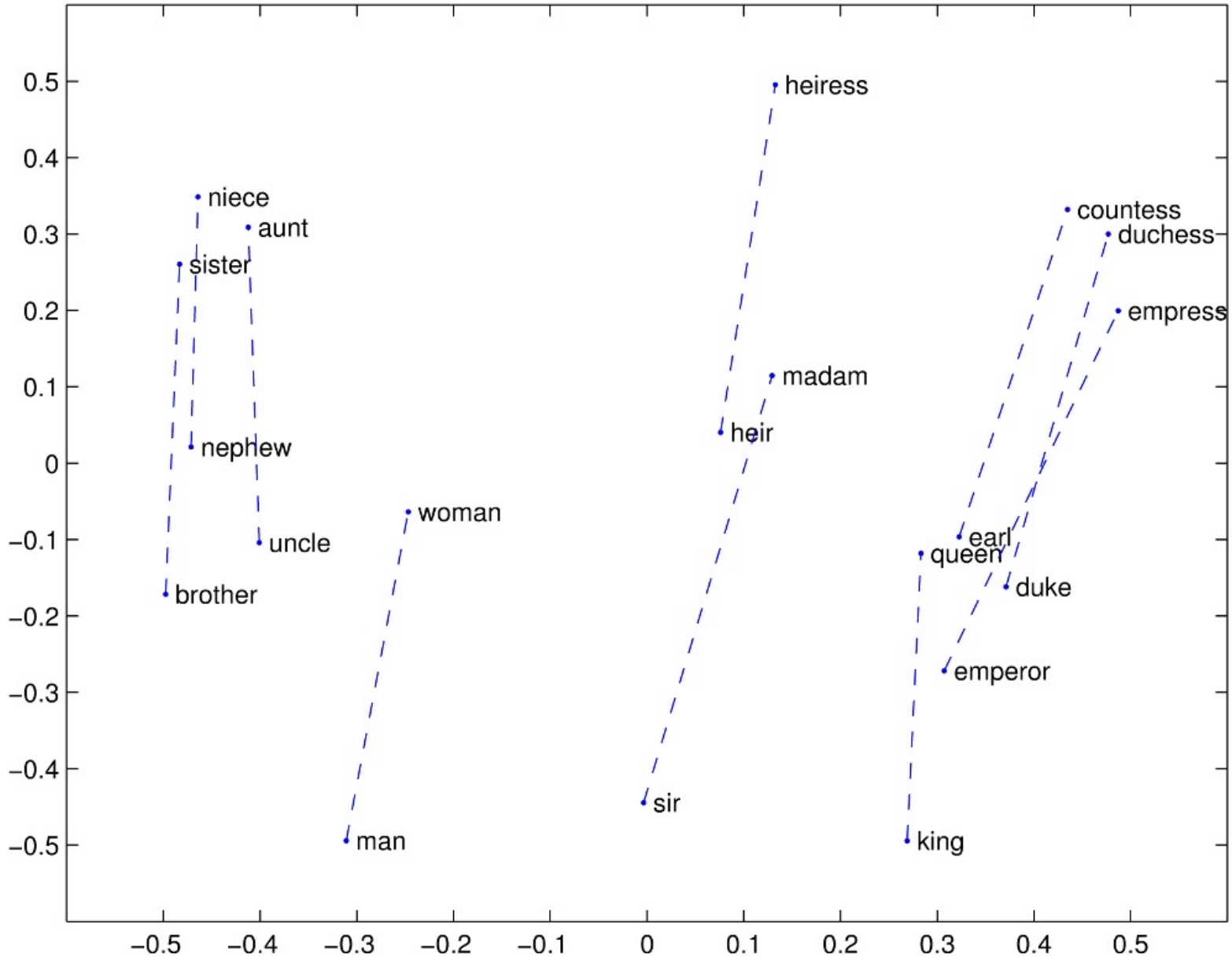
- GloVe apprende i vettori w_i in modo tale che

$$w_i \cdot w_j \approx \log P(w_i|w_j)$$

- Ne consegue che:

$$\log \frac{P(w_i|w_j)}{P(w_i|w_k)} = \log P(w_i|w_j) - \log P(w_i|w_k) \approx w_i \cdot (w_j - w_k)$$

Struttura dello spazio degli embedding GloVe

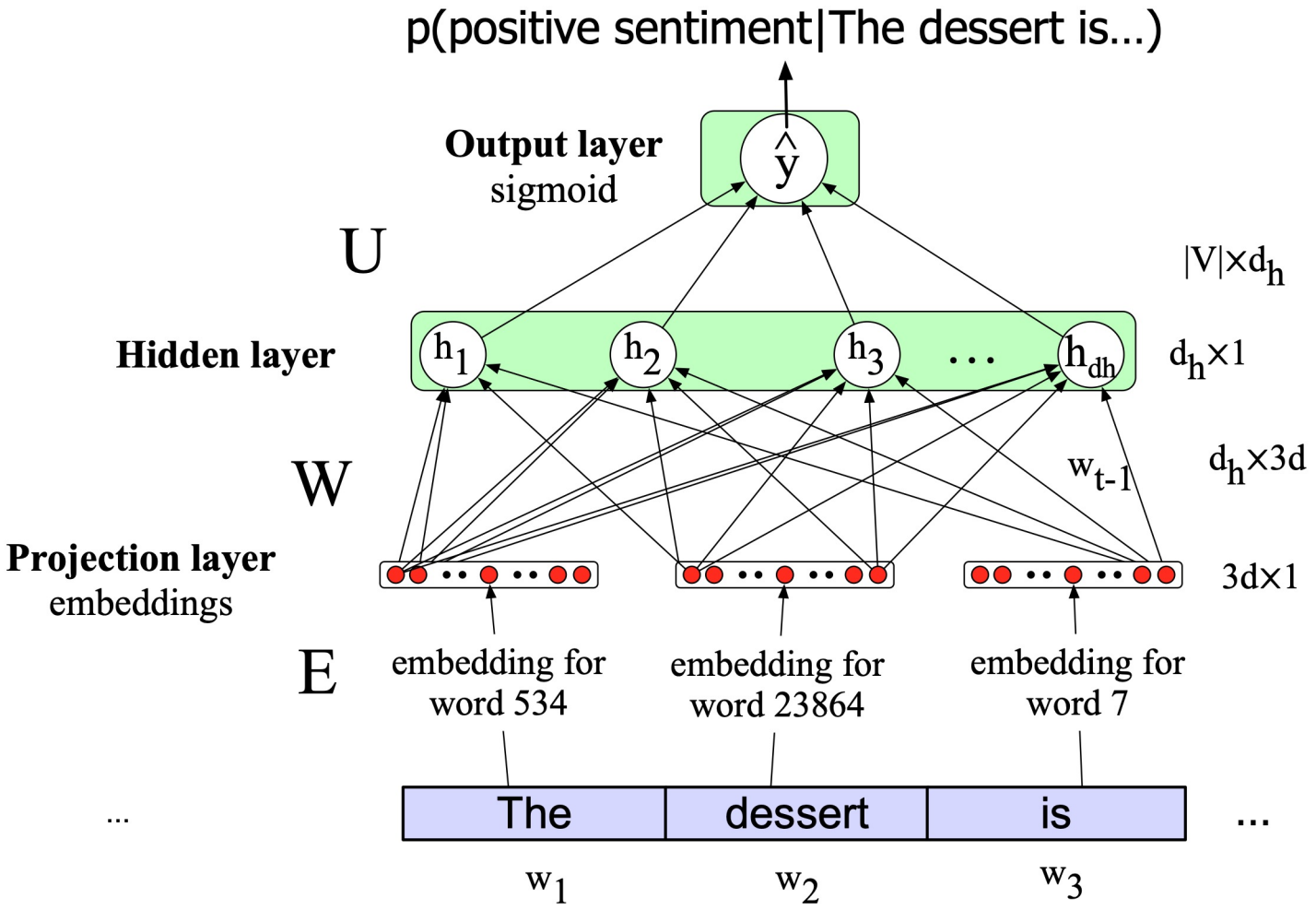


Language Model Neurali

Language Model neurali

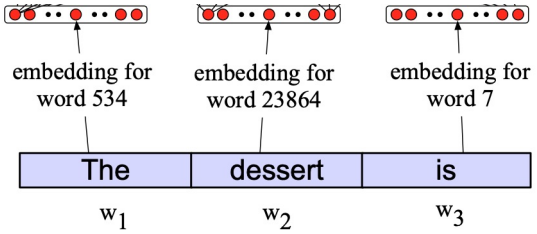
- *Language Modeling*: calcolare la probabilità di una parola in una sequenza, posto di averne osservate alcune.
- Abbiamo visto gli N-grammi
- I LM neurali li battono di gran lunga
- I LM neurali allo stato dell'arte sono basati su reti neurali molto potenti quali i Transformer
- Anche semplici modelli feedforward hanno una buona performance

Esempio: classificatore neurale con embedding come feature di ingresso



Problema: I testi hanno dimensioni differenti

- Il nostro esempio assume lunghezza fissa pari a tre
- Irrealistico
- Alcune semplici soluzioni



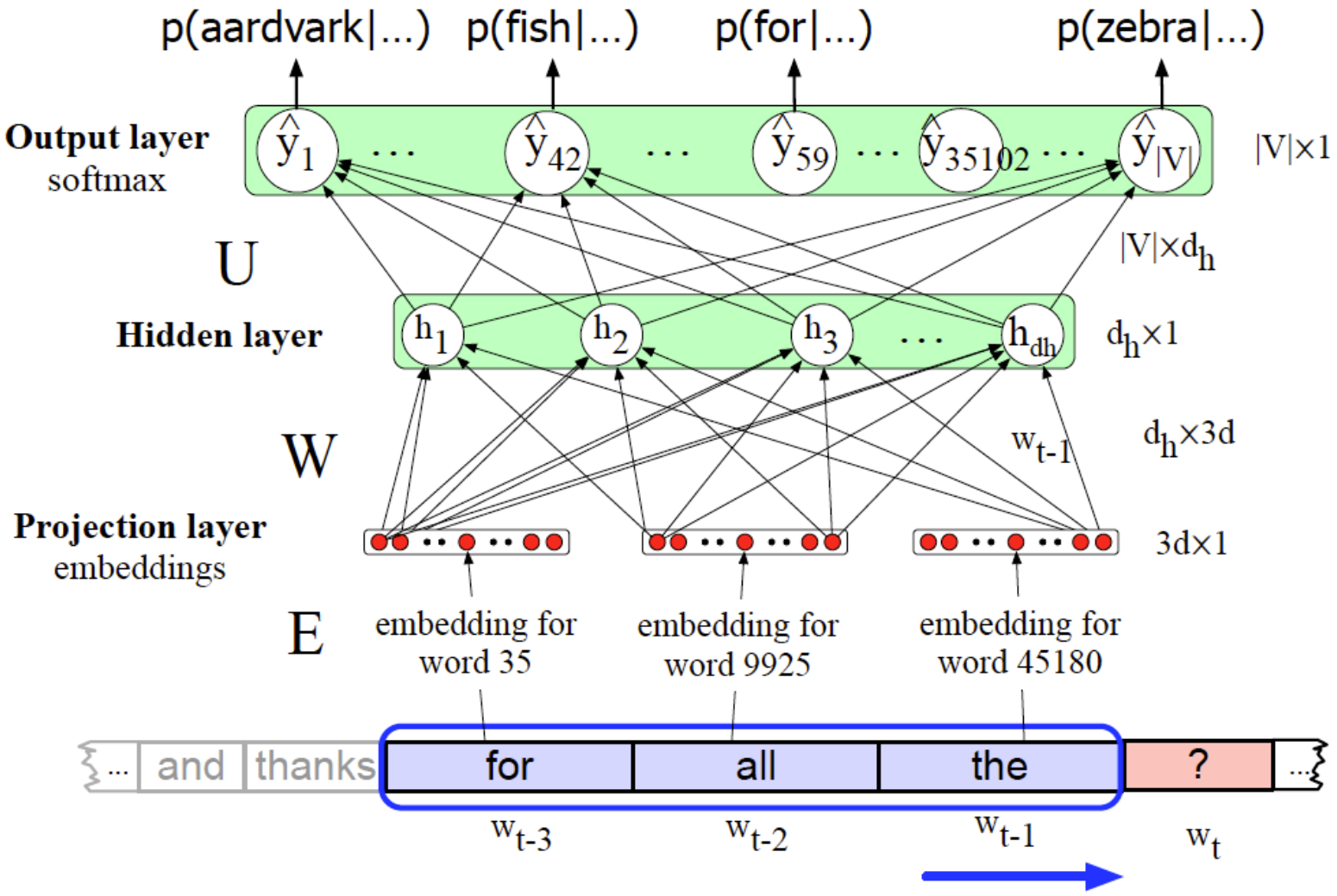
1. Creiamo un input lungo quanto la più luga sequenza osservata
   - Pad con 0 per sequenze più corte
   - Tronchiamo le sequenze di test più lunghe dell'input
2. Creiamo un unico "sentence embedding" della stessa dimensionalità dei word embedding
   - Consideriamo la media di tutti I word embedding
   - Consideriamo il massimo elemento per elemento di tutti I word embedding
     - Il massimo per ogni dimensione tra tutte le parole

Semplice neural language model feedforward

- **Task**: predire la parola successiva w_t
- date le parole precedenti w_{t-1}, w_{t-2}, w_{t-3}, …
- **Problema**: dobbiamo gestire sequenze di lunghezza arbitraria.
- **Soluzione**: Sliding windows (di lunghezza fissa)

$$P(w_t|w_1^{t-1}) \approx P(w_t|w_{t-N+1}^{t-1})$$

Neural Language Model



Università degli Studi di Palermo

CHILAB

Perché i LM neurali sono migliori degli N-grammi

- **Dati di training:**
- Abbiamo osservato: Devo esser certo che il gatto abbia mangiato
- Non abbiamo mai visto: il cane abbia mangiato

- **Dati di test:**
- Ho scordato di assicurarmi che il cane _____
- Il LM a N-grammi non può predire “abbia mangiato”!
- Il LM neurale usa la similarità tra gli embedding di “gatto” e “cane” per generalizzare e predire “abbia mangiato” dopo cane